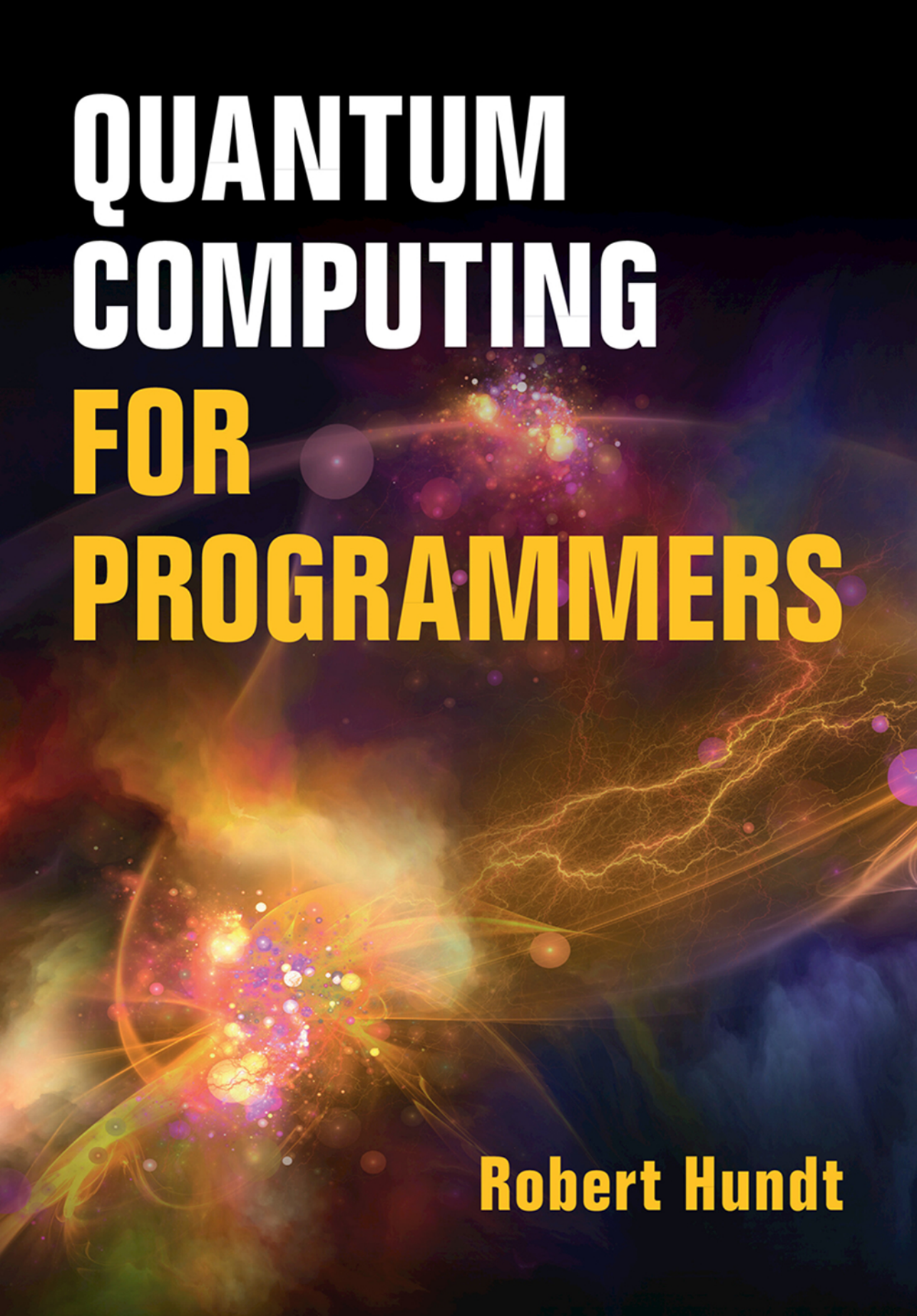




QUANTUM COMPUTING FOR PROGRAMMERS

Robert Hundt

Quantum Computing for Programmers

This introduction to quantum computing from a classical programmer's perspective is meant for students and practitioners alike. More than 25 fundamental algorithms are explained with full mathematical derivations and classical code for simulation, using an open-source code base developed from the ground up in Python and C++.

After presenting the basics of quantum computing, the author focuses on algorithms and the infrastructure to simulate them efficiently, beginning with quantum teleportation, superdense coding, Bernstein–Vazirani's algorithm, and Deutsch–Jozsa's algorithm. Coverage of advanced algorithms includes the quantum supremacy experiment, quantum Fourier transform, phase estimation, Shor's algorithm, Grover's algorithm with quantum counting and amplitude amplification, quantum random walks, and the Solovay–Kitaev algorithm for gate approximation. Quantum simulation is explored with the variational quantum eigensolver, quantum approximate optimization, and the NP-complete Max-Cut and Subset-Sum algorithms.

The book also discusses issues around programmer productivity, quantum noise, error correction, and challenges for quantum programming languages, compilers, and tools, with a final section on compiler techniques for transpilation.

Robert Hundt is a Distinguished Engineer at Google, where he led software development for Google's TPU supercomputers, the XLA compiler for TensorFlow, an open-source CUDA compiler, and currently the high-level synthesis toolchain XLS. He has more than 25 scientific publications, holds more than 35 patents, and is a senior member of IEEE.

Quantum Computing for Programmers

ROBERT HUNDT



CAMBRIDGE
UNIVERSITY PRESS

CAMBRIDGE
UNIVERSITY PRESS

University Printing House, Cambridge CB2 8BS, United Kingdom

One Liberty Plaza, 20th Floor, New York, NY 10006, USA

477 Williamstown Road, Port Melbourne, VIC 3207, Australia

314–321, 3rd Floor, Plot 3, Splendor Forum, Jasola District Centre,
New Delhi – 110025, India

103 Penang Road, #05–06/07, Visioncrest Commercial, Singapore 238467

Cambridge University Press is part of the University of Cambridge.

It furthers the University's mission by disseminating knowledge in the pursuit of education, learning, and research at the highest international levels of excellence.

www.cambridge.org

Information on this title: www.cambridge.org/9781009098175

DOI: [10.1017/9781009099974](https://doi.org/10.1017/9781009099974)

© Robert Hundt 2022

This publication is in copyright. Subject to statutory exception and to the provisions of relevant collective licensing agreements, no reproduction of any part may take place without the written permission of Cambridge University Press.

First published 2022

Printed in the United Kingdom by TJ Books Limited, Padstow Cornwall

A catalogue record for this publication is available from the British Library.

Library of Congress Cataloging-in-Publication Data

Names: Hundt, Robert, author.

Title: Quantum computing for programmers / Robert Hundt.

Description: Cambridge, United Kingdom ; New York, NY : Cambridge

University Press, 2022. | Includes bibliographical references and index.

Identifiers: LCCN 2021044761 (print) | LCCN 2021044762 (ebook) |

ISBN 9781009098175 (hardback) | ISBN 9781009099974 (epub)

Subjects: LCSH: Quantum computing. | BISAC: COMPUTERS / General

Classification: LCC QA76.889 .H86 2022 (print) | LCC QA76.889 (ebook) |

DDC 006.3/843–dc23/eng/20211105

LC record available at <https://lcn.loc.gov/2021044761>

LC ebook record available at <https://lcn.loc.gov/2021044762>

ISBN 978-1-009-09817-5 Hardback

Cambridge University Press has no responsibility for the persistence or accuracy of URLs for external or third-party internet websites referred to in this publication and does not guarantee that any content on such websites is, or will remain, accurate or appropriate.

To Mary, Thalia, and Johannes

Contents

	<i>Acknowledgments</i>	page x
	<i>Introduction</i>	xi
1	The Mathematical Minimum	1
	1.1 Complex Numbers	1
	1.2 Dirac Notation, Bras, and Kets	2
	1.3 Tensor Product	5
	1.4 Unitary and Hermitian Matrices	6
	1.5 Hermitian Adjoint of Expressions	6
	1.6 Eigenvalues and Eigenvectors	7
	1.7 Trace of a Matrix	8
2	Quantum Computing Fundamentals	9
	2.1 Tensors	9
	2.2 Qubits	13
	2.3 States	15
	2.4 Helper Functions	24
	2.5 Operators	25
	2.6 Single-Qubit Gates	33
	2.7 Controlled Gates	46
	2.8 Quantum Circuit Notation	51
	2.9 Bloch Sphere	55
	2.10 Global Phase	59
	2.11 Entanglement	60
	2.12 No-Cloning Theorem	65
	2.13 Uncomputation	66
	2.14 Reduced Density Matrix and Partial Trace	68
	2.15 Measurement	72
3	Simple Algorithms	78
	3.1 Random Number Generator	78
	3.2 Gate Equivalences	79
	3.3 Classical Arithmetic	89
	3.4 Swap Test	93

3.5	Quantum Teleportation	97
3.6	Superdense Coding	102
3.7	Bernstein–Vazirani Algorithm	105
3.8	Deutsch’s Algorithm	108
3.9	Deutsch–Jozsa Algorithm	118
4	Scalable, Fast Simulation	122
4.1	Simulation Complexity	122
4.2	Quantum Registers	124
4.3	Circuits	126
4.4	Fast Gate Application	134
4.5	Accelerated Gate Application	139
4.6	Sparse Representation	145
5	Beyond Classical	149
5.1	10,000 Years, 2 Days, or 200 Seconds	150
5.2	Quantum Random Circuit Algorithm	150
5.3	Circuit Construction	152
5.4	Estimation	155
5.5	Evaluation	158
6	Complex Algorithms	160
6.1	Phase Kick	161
6.2	Quantum Fourier Transform	163
6.3	Quantum Arithmetic	172
6.4	Phase Estimation	180
6.5	Shor’s Algorithm	189
6.6	Order Finding	196
6.7	Grover’s Algorithm	210
6.8	Amplitude Amplification	227
6.9	Quantum Counting	230
6.10	Quantum Random Walk	234
6.11	Variational Quantum Eigensolver	240
6.12	Quantum Approximate Optimization Algorithm	253
6.13	Maximum Cut Algorithm	254
6.14	Subset Sum Algorithm	262
6.15	Solovay–Kitaev Theorem and Algorithm	266
7	Quantum Error Correction	278
7.1	Quantum Noise	278
7.2	Quantum Error Correction	284
7.3	Nine-Qubit Shor Code	289

8	Quantum Languages, Compilers, and Tools	292
8.1	Challenges for Quantum Compilation	293
8.2	Quantum Programming Model	294
8.3	Quantum Programming Languages	294
8.4	Compiler Optimization	303
8.5	Transpilation	311
Appendix:	Sparse Implementation	322
	<i>References</i>	335
	<i>Index</i>	343

Acknowledgments

A book like this would not be possible without the help of many people. Vincent Russo and Timofey Golubev found a large number of issues with the mathematical formulation, code, and writing. Gabriel Hannon provided valuable pointers to related physics concepts. Several of my questions were answered on the Quantum Computing Stack Exchange, a very helpful resource and community. Wes Cowley and Sarah Schedler provided line editing, Sue Klefstad produced the impressive index, and Eleanor Bolton provided outstanding copyediting services. I am grateful to Tiago Leao for pointing me to Beauregard (2003), which was key for my implementation of Shor's refactoring. Together with Rui Maia, Tiago also provides the community with a much-appreciated reference implementation (Leao, 2021).

I also want to thank many of my colleagues at Google. Dave Bennet and Michael Dorner went through the first draft of this text, which, I am certain, was not a pleasant experience. Their feedback helped to shape this work into an actual learning resource. Two of my colleagues stand out for their rigor and obsession with details: Fedor Kostritsa, who provided most detailed comments on text, math, derivations, and code; and Ton Kalker, who diligently reviewed the whole text and helped greatly in sharpening the mathematical formulation. Sergio Boixo and Benjamin Villalonga corrected many of my misunderstandings about the quantum supremacy experiment. Michael Broughton and Craig Gidney helped to improve the section on Grover's algorithm. Craig also maintains the elegant online simulator Quirk. Thanks also to Mark Heffernan, Chris Leary, Rob Springer, and Mirko Rossini. Finally, I owe gratitude to Aamer Mahmood for his extraordinary support.

Without exception, my contacts at Cambridge University Press were outstanding. First and foremost, I must thank my editor, Lauren Cowles, who did a tremendous job guiding me throughout the whole process.

Finally, and most importantly, I am incredibly thankful to my family, including my dog, Charlie, for their love, patience, and support during this all-consuming effort.

Introduction

I think I can safely say that nobody understands quantum mechanics

Feynman (1965)

I have been impressed by numerous instances of mathematical theories that are really about particular algorithms; these theories are typically formulated in mathematical terms that are much more cumbersome and less natural than the equivalent formulation today's computer scientists would use

Knuth (1974)

This book is an introduction to quantum computing from the perspective of a classical programmer. Most concepts are explained with code, based on the insight that much of the complicated-looking math typically found in quantum computing may look quite simple in code. For many programmers, reading code is faster than reading complex math notation. Coding also allows experimentation, which helps in building intuition and understanding of the fundamental mechanisms of quantum computing. I believe that this approach will make it efficient and fun to get started.

Contrary to other learning resources, we will not use available software frameworks in this book, such as the well-developed Qiskit toolkit from IBM or Google's Cirq. Instead we build our own infrastructure from the ground up, based initially on Python's `numpy` library. It turns out that for learning the fundamentals, only a few hundred lines of code are required. This initial code is slow, but explicit. It is easy to debug and experiment with, which makes it an excellent learning vehicle.

We then improve the infrastructure, accelerate it with C++, and detail an elegant, sparse representation. We introduce basic compiler concepts that allow transpilation of our circuits to those other platforms – Qiskit, Cirq, and others. This enables the use of these systems' advanced features, such as scale-out performance and advanced error models.

Typically, an introduction to quantum computing is prefaced by a sizeable reintroduction of complex linear algebra. We will not follow this pattern here. Many programmers do have a solid foundation in linear algebra, but others lack the background or interest in this topic. It is my goal to be an attractive learning resource for both groups, without getting into the details of linear algebra. Hence I only assume basic familiarity with complex numbers, vectors, and matrices. We review a handful of core concepts in [Chapter 1](#). As we go along in the text we add a small number of additional

mathematical concepts that are necessary to understand the algorithms. We hope that this format will be helpful to the linear algebra-challenged while not being too shallow for the cognoscenti. After the introduction of the basic mathematical concepts, the book is organized into the following major sections:

- In [Chapter 2](#) we introduce core concepts of quantum computing and implement them with full matrices and vectors in Python. We discuss states, operators, entanglement, and measurement. We show a variety of ways to construct, describe, and analyze qubits and quantum circuits. Quantum mechanics, superposition, entanglement, and measurement are all complex and deeply philosophical topics. In this text, however, we focus exclusively on the computational aspects of the theory.
- This is followed in [Chapter 3](#) with introductory algorithms, utilizing the infrastructure developed so far. The material is presented in an elaborate way, with detailed mathematical derivations.
- The basic infrastructure developed up to this point does not scale to a higher number of qubits, which are needed for complex algorithms. In [Chapter 4](#), we detail techniques for faster gate application and accelerate with C++. We demonstrate how a sparse representation can produce the best performing results for a certain class of algorithms. These sections lead to building a high-performance quantum simulator. Readers not interested in infrastructure may skim or even skip this section.
- In [Chapter 5](#) we convince ourselves that quantum computers indeed have capabilities that go *beyond classical* machines.
- With this insight, [Chapter 6](#) on complex algorithms provides important details on several of quantum computing's core algorithms, including Grover's search, the quantum Fourier transform, phase estimation, quantum random walks, Shor's algorithm for integer factorization, and a variational quantum eigensolver with some applications. This section also details the seminal Solovay–Kitaev algorithm to approximate any unitary gate from a small universal gate set. The foundation built up in the earlier chapters is sufficient to implement and fully appreciate these marvelous algorithms.
- [Chapters 7](#) and [8](#) then address practical issues around programmer productivity. We touch on quantum error correction, which is essential to the feasibility of quantum computing. We also discuss quantum programming language design, compilers, and tools to further improve programmer productivity.
- The Appendix contains additional interesting material that did not fit the main text's flow. Specifically, it contains a detailed description of the sparse simulation infrastructure.

Source Code

Much of the content in this book is explained with both math and code. To avoid turning this book into a giant code listing, however, we abbreviate less interesting or

repetitive pieces of code with constructs such as `[...]`. Scaffolding code, such as Python `import` statements or C++ `#include` directives, are typically omitted. The full sources are hosted under a permissive Apache license on GitHub, along with instructions on how to download, build, and run:

<https://github.com/qcc4cp/qcc>

Contributions, comments, and suggestions are always welcome. Typesetting the code may have introduced errors, but the source of truth is the online repository. The code is also likely to have evolved beyond what is published here.

1 The Mathematical Minimum

In this section, we briefly discuss the minimum mathematical background that is required to fully understand this text. Readers who are familiar with the concepts may safely skip this section. Readers who are easily discouraged by even basic math may proceed to the [next chapter](#) and refer back here later.

1.1 Complex Numbers

Let us briefly recapitulate complex numbers. A complex number z is of the form

$$z = x + iy.$$

The x is called the real part of z ; y is the imaginary part. The imaginary number i is defined as the solution to the equation:

$$x^2 + 1 = 0.$$

In other words, i is defined as the square root of -1 . A complex number's conjugate, often denoted by $\bar{z}$ or z^* , is created by simply negating its imaginary part: $i \rightarrow -i$. For example, for $z = 5 + 2i$ the conjugate z^* would simply be $z^* = 5 - 2i$.

The conjugate of a product of complex numbers is equal to the product of the conjugates of the complex numbers:

$$(ab)^* = a^*b^*.$$

The *norm* of a complex number, denoted by $|z|$, is computed by multiplying the complex number with its conjugate:

$$|z|^2 = z^*z,$$

$$|z| = \sqrt{z^*z}.$$

Complex numbers can be drawn in the 2D plane with an x- and y-axis according to the definition. If we think of a complex number as a vector originating at $(0, 0)$, the norm of a complex number, which is then the length of the corresponding vector, is a real number and can be computed using Pythagoras' theorem as:

$$|z| = |x + iy| = \sqrt{(x - iy)(x + iy)} = \sqrt{x^2 + y^2}$$

For complex numbers, the norm is commonly referred to as the *modulus*. Note the difference between the *square* of a complex number and its squared *norm*. The square is computed as:

$$z^2 = (x + iy)^2 = (x + iy)(x + iy) = x^2 + 2ixy - y^2.$$

Complex exponentiation is defined by Euler's famous formula:

$$re^{i\phi} = r(\cos(\phi) + i\sin(\phi)).$$

Correspondingly, for complex numbers with norm $|z| = r = 1.0$:

$$z = e^{i\phi} = \cos(\phi) + i\sin(\phi).$$

The resulting complex numbers from this exponentiation are on a unit circle around the origin $(0, 0)$.

In Python, complex numbers are, conveniently, part of the language. Note however, that the imaginary i is written as a `j`, which is customarily used in electrical engineering. An example:

```
x = 1.0 + 0.5j
```

To conjugate, you can use the built-in `conjugate()` function for the complex data types or use `numpy`'s `conj()` function. For example:

```
x_conj = x.conjugate()    # or
x_conj = np.conj(x)
```

1.2 Dirac Notation, Bras, and Kets

In quantum computing, we think of qubits and states as column vectors of n complex numbers, where n is typically a power of 2. A vector with n elements is called n -dimensional. In the so-called *Dirac* notation, a column vector is called a *ket* and written as $|x\rangle$:

$$|x\rangle = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix}, \text{ with } x_i \in \mathbb{C} \text{ and } |x\rangle \in \mathbb{C}^n.$$

Remember that to *transpose* a matrix A , we take column i of A and make it row i of the transpose A^T , or $A_{ij}^T = A_{ji}$. The *Hermitian conjugate* of a column vector $|x\rangle$,

denoted by a dagger $|x\rangle^\dagger$, is the transpose of the vector with each element conjugated. We write this vector as $\langle x|$, changing the direction of the angle bracket:

$$|x\rangle^\dagger = \langle x| = [x_0^* \quad x_1^* \quad \dots \quad x_{n-1}^*].$$

In Dirac notation, such a row vector $\langle x|$ is called a *bra* or the *dual vector* for a ket $|x\rangle$. Transposition and conjugation goes both ways – applying the transformation twice results in the original ket, a property called *involutivity*.

$$\begin{aligned} |x\rangle^\dagger &= \langle x|, \\ \langle x|^\dagger &= |x\rangle, \\ (|x\rangle^\dagger)^\dagger &= |x\rangle. \end{aligned}$$

There is potential for confusion around the conjugates: should the conjugates be denoted explicitly, via a^* or $a^\dagger$, as in $\langle x_0^* \ x_1^* \ \dots \ x_{n-1}^*|$, or is the fact that a vector has been converted from ket to bra sufficient? Typically, the conjugates are *not* marked explicitly.

1.2.1 Inner Product

The inner product, which is also called the *scalar product* or the *dot product*, is computed as a matrix product of a bra and a ket, which simplifies to the product between a row vector and a column vector – an element-wise vector-vector multiplication and summation. It is written in the following forms, with the dot ($\cdot$) denoting a scalar product:

$$\langle x| \cdot |y\rangle = \langle x||y\rangle = \langle x|y\rangle.$$

For kets $|x\rangle$ and $|y\rangle$, the inner product is defined as:

$$|x\rangle = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix}, \quad \langle x| = [x_0^* \quad x_1^* \quad \dots \quad x_{n-1}^*], \quad |y\rangle = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix},$$

$$\langle x|y\rangle = x_0^*y_0 + x_1^*y_1 + \dots + x_{n-1}^*y_{n-1}.$$

The inner product is how vectors in this notation get their names. It forms a product of a bra and a ket, a *bra(c)ket*. Naming is difficult in general and quantum computing is no exception.

Note that $\langle x|y\rangle$ does not generally equal $\langle y|x\rangle$. For example, consider two kets $|x\rangle$ and $|y\rangle$:

$$|x\rangle = \begin{bmatrix} -1 \\ 2i \\ 1 \end{bmatrix}, \quad |y\rangle = \begin{bmatrix} 1 \\ 0 \\ i \end{bmatrix}. \quad (1.1)$$

We construct the corresponding bras via transposition and negation of the imaginary parts:

$$\langle x| = [-1 \quad -2i \quad 1], \quad \langle y| = [1 \quad 0 \quad -i].$$

We then compute the inner products:

$$\langle x|y\rangle = -1 * 1 + 2i * 0 + i * 1 = -1 + i,$$

$$\langle y|x\rangle = 1 * -1 + 0 * 2i - i * 1 = -1 - i.$$

The second result is the conjugate of the first; the two inner products are different. This points to the important general rule:

$$\langle x|y\rangle^* = \langle y|x\rangle.$$

Two vectors are *orthogonal* if and only if their scalar product is zero. For 2D vectors, we visualize orthogonal vectors as perpendicular to each other:

$$\langle x|y\rangle = 0 \Rightarrow x, y \text{ orthogonal}.$$

Similar to the way in which we compute the norm of a complex number, the norm of a vector is the scalar product of the vector with its dual vector. A vector is *normalized* if its norm is 1:

$$||x\rangle| = \langle x|x\rangle = 1 \Rightarrow |x\rangle \text{ normalized.} \quad (1.2)$$

State vectors in quantum computing represent probability distributions that must total 1.0 by definition. Hence, normalized vectors play an important role in quantum computing.

1.2.2 Outer Product

Corresponding to the inner product, we can construct an *outer* product between two kets $|x\rangle$ and $|y\rangle$, denoted as:

$$|x\rangle\langle y| = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} \begin{bmatrix} y_0^* & y_1^* & \cdots & y_{n-1}^* \end{bmatrix} = \begin{bmatrix} x_0 y_0^* & x_0 y_1^* & \cdots & x_0 y_{n-1}^* \\ x_1 y_0^* & x_1 y_1^* & \cdots & x_1 y_{n-1}^* \\ \vdots & \vdots & \ddots & \vdots \\ x_{n-1} y_0^* & x_{n-1} y_1^* & \cdots & x_{n-1} y_{n-1}^* \end{bmatrix}.$$

In the example given by [Equation \(1.1\)](#), $|x\rangle$ is a 3×1 vector and $|y\rangle$ is a 1×3 vector. By the rules of matrix multiplication, their outer product will be a 3×3 matrix. Again, if the vector elements are complex, we conjugate the vector elements when converting from bra to ket and vice versa.

1.3 Tensor Product

To compute the *tensor product*¹ of two vectors, which can be either bras or kets, we use any of these notations:

$$|x\rangle \otimes |y\rangle = |x\rangle|y\rangle = |x, y\rangle = |xy\rangle. \quad (1.3)$$

And correspondingly:

$$\langle x| \otimes \langle y| = \langle x|\langle y| = \langle x, y| = \langle xy|.$$

In a tensor product, each element of the first constituent is multiplied with the whole of the second constituent. Hence, an $n \times m$ matrix tensored with an $k \times l$ matrix will result in an $nk \times ml$ matrix. For example, to compute the tensor products of the following two kets:

$$\begin{aligned} |0\rangle &= \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \\ |0\rangle \otimes |1\rangle &= |01\rangle = \begin{bmatrix} 1 & \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\ 0 & \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}. \end{aligned}$$

You can see that the tensor product of two kets is a ket. Similarly, the tensor product of two bras is a bra, and the tensor product of two diagonal matrices is a diagonal matrix. Of course, tensor products are also defined for general matrices:

$$\begin{aligned} \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} \otimes \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix} &= \begin{bmatrix} a_{00} \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix} & a_{01} \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix} \\ a_{10} \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix} & a_{11} \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix} \end{bmatrix} \\ &= \begin{bmatrix} a_{00}b_{00} & a_{00}b_{01} & a_{01}b_{00} & a_{01}b_{01} \\ a_{00}b_{10} & a_{00}b_{11} & a_{01}b_{10} & a_{01}b_{11} \\ a_{10}b_{00} & a_{10}b_{01} & a_{11}b_{00} & a_{11}b_{01} \\ a_{10}b_{10} & a_{10}b_{11} & a_{11}b_{10} & a_{11}b_{11} \end{bmatrix}. \end{aligned}$$

For multiplication of scalars α and β with a tensor product, these rules apply:

$$\alpha(x \otimes y) = (\alpha x) \otimes y = x \otimes (\alpha y), \quad (1.4)$$

$$(\alpha + \beta)(x \otimes y) = \alpha x \otimes y + \beta x \otimes y. \quad (1.5)$$

A key property of the the tensor product is the following – it is used in many derivations in this text:

$$(A \otimes B)(a \otimes b) = (A \otimes a)(B \otimes b). \quad (1.6)$$

¹ Here, we are ignoring differences between the tensor product and the Kronecker product.

The next rule is also very important for this text. Given two *composite* kets:

$$|\psi_1\rangle = |\phi_1\rangle \otimes |\chi_1\rangle \quad \text{and} \quad |\psi_2\rangle = |\phi_2\rangle \otimes |\chi_2\rangle,$$

the inner product between $|\psi_1\rangle$ and $|\psi_2\rangle$ is computed as:

$$\begin{aligned} \langle\psi_1|\psi_2\rangle &= (|\phi_1\rangle \otimes |\chi_1\rangle)^\dagger (|\phi_2\rangle \otimes |\chi_2\rangle) \\ &= (\langle\phi_1| \otimes \langle\chi_1|)(|\phi_2\rangle \otimes |\chi_2\rangle) \\ &= \langle\phi_1|\phi_2\rangle \langle\chi_1|\chi_2\rangle. \end{aligned} \tag{1.7}$$

1.4 Unitary and Hermitian Matrices

A square matrix A is *Hermitian* if it is equal to its transposed complex conjugate $A^\dagger$. Hence the diagonal elements must be real numbers, and the elements mirrored along the main diagonal are the complex conjugates of each other. For example:

$$A = A^\dagger = \begin{bmatrix} 1 & 3 + i\sqrt{2} \\ 3 - i\sqrt{2} & 0 \end{bmatrix}.$$

A square matrix A is *unitary* if its conjugate transpose is equal to its inverse, with $A^\dagger A = I$. Unitary matrices are *norm preserving* – multiplying a unitary matrix with a vector might change the vector's orientation but will not change its norm. For example, here the matrix Y is both unitary and Hermitian. The matrix S is unitary but *not* Hermitian:

$$Y = Y^\dagger = \begin{bmatrix} 0 & i \\ -i & 0 \end{bmatrix}, \quad \text{and} \quad S = \begin{bmatrix} 1 & 0 \\ 0 & e^i \end{bmatrix} \neq \begin{bmatrix} 1 & 0 \\ 0 & e^{-i} \end{bmatrix} = S^\dagger.$$

Similar to the way we computed Hermitian conjugates for vectors in [Section 1.2](#), to construct the Hermitian *conjugate* of a square matrix, you have to transpose the matrix and conjugate its elements. A Hermitian conjugate is also called a Hermitian *adjoint*, or just adjoint for short. The terms adjoint and Hermitian conjugate are synonyms.

1.5 Hermitian Adjoint of Expressions

Here are the rules for how to conjugate expressions of matrices and vectors. We have already learned how to convert between bras and kets:

$$\begin{aligned} |\psi\rangle^\dagger &= \langle\psi|, \\ \langle\psi|^\dagger &= |\psi\rangle. \end{aligned}$$

To compute the adjoint of a matrix scaled by a complex factor:

$$(\alpha A)^\dagger = \alpha^* A^\dagger. \tag{1.8}$$

For matrix-matrix products, the order reverses (this is an important rule used in this book):

$$(AB)^\dagger = B^\dagger A^\dagger. \quad (1.9)$$

And similarly, to compute the adjoint for products of matrices and vectors:

$$(A|\psi\rangle)^\dagger = \langle\psi|A^\dagger, \quad (1.10)$$

$$(AB|\psi\rangle)^\dagger = \langle\psi|B^\dagger A^\dagger. \quad (1.11)$$

For matrices in outer product notation, this rule is easy to derive:

$$A = |\psi\rangle\langle\phi| \Rightarrow A^\dagger = |\phi\rangle\langle\psi|. \quad (1.12)$$

And finally:

$$(A + B)^\dagger = A^\dagger + B^\dagger. \quad (1.13)$$

1.6 Eigenvalues and Eigenvectors

There is a special case of matrix-vector multiplication, where the following equation holds. Here, A is a square matrix, $|\psi\rangle$ is a ket, and λ is a simple (complex) scalar:

$$A|\psi\rangle = \lambda|\psi\rangle.$$

Applying A to the special vector $|\psi\rangle$ only *scales* the vector with a complex number, it does not change its orientation. We call λ an *eigenvalue* of A . There can be multiple eigenvalues for a given operator. The corresponding vectors for which this equation holds are called *eigenvectors*. In quantum mechanics, the synonym *eigenstates* is also used. Eigenvalues are allowed to be 0 by definition, but a null vector is not considered an eigenvector.

Diagonal matrices are a case for which finding the eigenvalues is trivial. Given a diagonal matrix of this form:

$$\begin{bmatrix} \lambda_0 & & & \\ & \lambda_1 & & \\ & & \ddots & \\ & & & \lambda_{n-1} \end{bmatrix},$$

we can pick the eigenvalues right off the diagonal. The corresponding eigenvectors are the computational bases $(1, 0, 0, \dots)^T$, $(0, 1, 0, \dots)^T$, and so on. For Hermitian matrices the eigenvalues are necessarily real.

1.7 Trace of a Matrix

The *trace* of an $n \times n$ matrix A is defined as the sum of its diagonal elements:

$$\text{tr}(A) = \sum_{i=0}^{n-1} a_{ii} = a_{00} + a_{11} + \cdots + a_{n-1,n-1}.$$

Basic properties of the trace are the following, where c is a scalar, and A and B are square matrices:

$$\text{tr}(A + B) = \text{tr}(A) + \text{tr}(B), \quad (1.14)$$

$$\text{tr}(cA) = c \text{tr}(A), \quad (1.15)$$

$$\text{tr}(AB) = \text{tr}(BA). \quad (1.16)$$

For tensor products, this important relation holds:

$$\text{tr}(A \otimes B) = \text{tr}(A) \text{tr}(B). \quad (1.17)$$

The trace of a Hermitian matrix is real because the diagonal elements of a Hermitian are real. The trace of a matrix A is the sum of its n eigenvalues λ_i :

$$\text{tr}(A) = \sum_{i=0}^{n-1} \lambda_i. \quad (1.18)$$

This next relation is important for measurements. Suppose we have two kets $|x\rangle$ and $|y\rangle$, such that

$$|x\rangle = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} \quad \text{and} \quad |y\rangle = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix}.$$

The trace of the outer product of $|x\rangle$ and $\langle y|$ is equal to their inner product:

$$\text{tr}(|x\rangle\langle y|) = \langle y|x\rangle. \quad (1.19)$$

This is easy to see from the outer product:

$$\begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} \begin{bmatrix} y_0^* & y_1^* & \cdots & y_{n-1}^* \end{bmatrix} = \begin{bmatrix} x_0 y_0^* & x_0 y_1^* & \cdots & x_0 y_{n-1}^* \\ x_1 y_0^* & x_1 y_1^* & \cdots & x_1 y_{n-1}^* \\ \vdots & \vdots & \ddots & \vdots \\ x_{n-1} y_0^* & x_{n-1} y_1^* & \cdots & x_{n-1} y_{n-1}^* \end{bmatrix}$$

$$\implies \text{tr}(|x\rangle\langle y|) = \sum_{i=0}^{n-1} x_i y_i^* = \langle y|x\rangle.$$

2 Quantum Computing Fundamentals

In this chapter, we describe the fundamental concepts and rules of quantum computing. In parallel, we develop an initial, easy-to-understand, and easy-to-debug code base for building and simulating smaller-scale algorithms.

The chapter is structured as follows. We first introduce our basic underlying data type, the Python `Tensor` type, which is derived from `numpy`'s `ndarray` data structure. Using this type we construct single qubits and quantum states composed of many qubits. We define operators, which allow us to modify states, and describe a range of important single-qubit gates. Controlled gates, which play a similar role to that of control flow in classical computing, come next. We detail how to describe quantum circuits via the Bloch sphere and in quantum circuit notation. A discussion of entanglement follows, that fascinating “spooky action at a distance,” as Einstein called it. In quantum physics, measurement might be even more problematic than entanglement (Norsen, 2017). In this text, we avoid philosophy and conclude the chapter by describing a simple way to simulate measurements.

2.1 Tensors

Quantum mechanics and quantum computing are expressed in the language of linear algebra – vectors, matrices, and operations such as the dot product, outer product, and Kronecker products. As we develop the theory, we complement it with working code to allow experimentation.

We start by describing a fundamental data structure in Python. Python may be slow in general, but it also has the vectorized and accelerated `numpy` numerical library for scientific computing. We will make heavy use of this library so that we do not have to implement standard numerical linear algebra operations ourselves. In general, we follow Google's coding style guides for Python (Google, 2021b) and C++ (Google, 2021a).

The core data types, such as states, operators, and density matrices, are all vectors and matrices of complex numbers. It is good practice to base all of them on one common type abstraction, which hides the underlying implementation. This approach avoids potential problems with type mismatches and makes analysis, testing, debugging, pretty-printing, and other common functionality easier to maintain consistently. The base data type for all subsequent work will be a common `Tensor` class.

We derive `Tensor` from the `ndarray` array data structure in `numpy`. Since we derive `Tensor`, it will behave just like a `numpy` array, but we can augment it with additional convenience functions.

There are several complex ways to instantiate an `ndarray`. The proper way to derive a class from this data type is complicated but well documented.¹ The implementation is in the open-source repository in `lib/tensor.py`:

```
import numpy as np

class Tensor(np.ndarray):
    """Tensor is a numpy array representing a state or operator."""

    def __new__(cls, input_array) -> Tensor:
        return np.asarray(input_array, dtype=tensor_type()).view(cls)

    def __array_finalize__(self, obj) -> None:
        if obj is None: return

        # np.ndarray has complex construction patterns. Because of this,
        # if new attributes are needed, add them like this:
        #     self.info = getattr(obj, 'info', None)
```

Note the use of `tensor_type()` in this code snippet: It abstracts the floating-point representation for complex numbers. Why do we do this? The choice of which complex data type to use is an interesting question, and each has its implications. Should it be complex numbers based on 64-bit doubles, 32-bit floats, or something else, for example, TPU's 16-bit bfloat format? Smaller data types are faster to simulate because of lower memory bandwidth requirements. But what level of accuracy is needed for which circuit? The `numpy` package supports `np.complex128` and `np.complex64`, so we simply define a global variable that holds this type's width. Having this information in one place makes it easy to experiment with different data types later on.

```
# Bit width of complex data type, 64 or 128.
tensor_width = 64

# All math in this package will use this base type.
# Valid values can be np.complex128 or np.complex64
def tensor_type():
    """Return complex type."""

    if tensor_width == 64:
        return np.complex64
    return np.complex128
```

¹ <https://numpy.org/doc/stable/user/basics.subclassing.html>.

As we will see in our discussion of quantum states in [Section 2.3](#), the Kronecker product of tensors, denoted with operator $\otimes$, is an important operation. This product is often referred to as *tensor product*. The *Kronecker product* describes a block product between matrices and is the correct term to use. We use the terms Kronecker product and tensor product interchangeably – *tensoring states* rolls off the tongue much more easily than *Kroneckering states*.

We implement it by adding the member function `kron` to the `Tensor` class. The function simply delegates to the function of the same name in `numpy`. We make heavy use of this operation, so for convenience we additionally overload the `*` operator to call this function.

There is the potential to confuse the `*` operator with simple matrix multiplication. However, in Python and with `numpy`, matrix multiplication is done with the *at* operator `@`. We inherit this operator from `numpy`; we don't have to implement it ourselves.

```
def kron(self, arg: Tensor) -> Tensor:
    """Return the Kronecker product of this object with arg."""

    return self.__class__(np.kron(self, arg))

def __mul__(self, arg: Tensor) -> Tensor:
    """Inline * operator maps to Kronecker product."""

    return self.kron(arg)
```

In our initial approach to quantum computing, we will often construct larger matrices by tensoring together many identical matrices, which corresponds to calling the `kron` function multiple times. For example, to tensor together n unitary matrices U , we will use the following notation:

$$\underbrace{U \otimes U \otimes \dots \otimes U}_n = U^{\otimes n}.$$

This is equivalent to a power function, but instead of multiplication we want Kronecker products. Naming is hard, but this function basically names itself. We should call it Kronecker power function – or `kpow` (pronounced “Ka-Pow”). We handle cases where the exponent is 0 as a special case, as $x^0 = 1.0$. As expected, `numpy` computes the correct Kronecker product of a matrix with a scalar:

```
def kpow(self, n: int) -> Tensor:
    """Return the tensor product with itself `n` times."""

    if n == 0:
        return 1.0
    t = self
    for _ in range(n - 1):
        t = np.kron(t, self)
    return self.__class__(t) # Necessary to return a Tensor type
```

Often, especially during testing, we want to compare a `Tensor` to a given value. We are working with complex numbers, which are based on the double or float data types. Direct comparison of values of these types is bad practice due to floating-point precision issues. Instead, for equality, we have to check that the difference between two numerical values is smaller than a given ϵ . Fortunately, `numpy` offers the function `allclose()`, which compares full tensors, so we do not have to iterate over dimensions and compare the real and imaginary parts. We use a tolerance of 10^{-6} , here and in almost all other places, and add this method to our `Tensor` type:²

```
def is_close(self, arg) -> bool:
    """Check that a 1D or 2D tensor is numerically close to arg."""

    return np.allclose(self, arg, atol=1e-6)
```

Python's `math` module has an `isclose` function. However, we decided to follow Google's coding style, which requires naming those types of function with an underscore after the `is_`. This is what we have chosen to do here to keep the overall code consistent with the style guide.

In [Section 1.4](#), we learned about Hermitian and unitary matrices. The two helper functions below check for these conditions:

```
def is_hermitian(self) -> bool:
    """Check if this tensor is Hermitian - Udag = U."""

    if len(self.shape) != 2:
        return False
    if self.shape[0] != self.shape[1]:
        return False
    return self.is_close(np.conj(self.transpose()))

def is_unitary(self) -> bool:
    """Check if this tensor is unitary - Udag*U = I."""

    return Tensor(np.conj(self.transpose()) @ self).is_close(
        Tensor(np.eye(self.shape[0])))
```

Some of the matrices we will encounter later in the text are *permutation matrices*, which have only a single 1 in each row and column of the matrix. This routine verifies this property:

² Note that for scalars, `math.isclose` is significantly faster than `np.allclose`. We will use it in performance-critical code.

```
def is_permutation(self) -> bool:
    x = self
    return (x.ndim == 2 and x.shape[0] == x.shape[1] and
            (x.sum(axis=0) == 1).all() and
            (x.sum(axis=1) == 1).all() and
            ((x == 1) or (x == 0)).all())
```

2.2 Qubits

In classical computing, a bit can have the value 0 or 1. If we think of a bit as a switch, it is either off or on. You could say it is in the off-state (0-state) or on-state (1-state). Quantum bits, which are also called *qubits*, can be in a 0-state or a 1-state as well. What makes them quantum is that they are in a superposition of these states: They can be in the 0-state and the 1-state at the same time. What does this mean exactly?

First, we have to distinguish between a *qubit* and the *state of a qubit*. Physical qubits, developed for real quantum computers, are real physical entities, such as ions captured in an electric field, Josephson junctions on an ASIC, and so on. The state of a qubit describes some measurable property of that qubit, for example, the energy level of an electron. In quantum computing, at the level of programming abstractions, the physical implementation does not matter; instead, we are concerned with the measurable state. This is similar to classical computing, where very few people are knowledgeable about the quantum effects that enable transistors at the level of logic gates. Thus, the terms qubit and state of the qubit are used interchangeably; we typically only use the term qubit.

This state space of one or more qubits is often denoted by the Greek symbol $|\psi\rangle$ (“psi”). The standard notation for a qubit’s 0-state is $|0\rangle$ in the Dirac notation. The 1-state is correspondingly written as $|1\rangle$. You can think of these as physically distinguishable states, such as the spin of an electron. These two states, $|0\rangle$ and $|1\rangle$, are known as *basis states*. We will *not* delve into the typical elaborate discussion of linear algebra and the theory of vector spaces here. All we need to know is that basis states represent orthogonal sets of vectors of dimensionality n (vectors with n components). Any vector of the same dimensionality can be constructed from linear combinations of basis states. In our context, we also require that basis vectors are normalized, forming an *orthonormal* set of basis vectors. Another way to say this is that the basis vectors are linearly independent and have a modulus of 1.

Superposition now simply means that the state of a qubit is a *linear combination* of orthonormal basis states, for example the $|0\rangle$ and $|1\rangle$ states:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

where α and β are complex numbers, called the *probability amplitudes*, with $|\alpha|^2 + |\beta|^2 = 1$.

Note that we use the square of the norm, not the square of a complex number. As will become clear later, this follows from one of the fundamental postulates of quantum mechanics: On measurement, the state collapses to either $|0\rangle$ with (real) probability $|\alpha|^2$ or $|1\rangle$ with (real) probability $|\beta|^2$. The state has to collapse to one of the two, and thus, the added probabilities must add up to 1.0. If both α and β are exactly $\sqrt{1/2}$, there is an equal $\sqrt{1/2}^2 = 1/2$ probability that the state collapses to $|0\rangle$ or $|1\rangle$ on measurement. If α is 1.0 and β is 0.0, it is certain that the state will collapse to the $|0\rangle$ state on measurement.

Let us look at a standard example. Given a qubit $|\phi\rangle$ as

$$|\phi\rangle = \frac{\sqrt{3}}{2} |0\rangle + \frac{i}{2} |1\rangle,$$

the probability of measuring $|0\rangle$ is

$$Pr_{|0\rangle}(|\phi\rangle) = \left(\frac{\sqrt{3}}{2}\right) \left(\frac{\sqrt{3}}{2}\right) = \frac{3}{4}.$$

The probability of measuring $|1\rangle$ is the following – we compute the norm squared of the factor $i/2$:

$$Pr_{|1\rangle}(|\phi\rangle) = \left|\frac{i}{2}\right|^2 = \left(\frac{-i}{2}\right) \left(\frac{i}{2}\right) = \frac{1}{4}.$$

The following code will translate these concepts into a straightforward implementation. As a forward reference, we use the type `State`, which we discuss in the next section. Put simply, a `State` is a vector of complex numbers implemented using `Tensor`.

To construct a qubit, we need either α or β , or both. If only one is provided, we can easily compute the other one, given that their squared norms must add to 1.0. To compute the norms of the complex numbers α and β , we multiply each one with its complex conjugate (hence the use of `np.conj()`). The result will be a real number. For the code not to generate a type error from `numpy`, we have to explicitly convert the result with `np.real()`. We compare the results to 1.0, and if it is within tolerance, we construct and return the qubit.

What data structure should we use to represent a qubit? We simply create an array of two complex values, fill in α and β , and return a `State` constructed from this array.

```
def qubit(alpha: Optional[np.complexfloating] = None,
          beta: Optional[np.complexfloating] = None) -> State:
    """Produce a given state for a single qubit."""

    if alpha is None and beta is None:
        raise ValueError('alpha, or beta, or both are required')

    if beta is None:
        beta = math.sqrt(1.0 - np.conj(alpha) * alpha)
```

```

if alpha is None:
    alpha = math.sqrt(1.0 - np.conj(beta) * beta)

if not math.isclose(np.conj(alpha) * alpha +
                    np.conj(beta) * beta, 1.0):
    raise ValueError('Qubit probabilities do not add to 1.')

qb = np.zeros(2, dtype=tensor.tensor_type())
qb[0] = alpha
qb[1] = beta
return State(qb)

```

From this code, you can infer what the basis states might look like – a state is just a complex vector: the $|0\rangle$ state should be $[1, 0]^T$ and the $|1\rangle$ state $[0, 1]^T$. With this in mind, the state of a qubit can be written in these forms:

$$\begin{aligned}
 |\psi\rangle &= \alpha|0\rangle + \beta|1\rangle \\
 &= \alpha \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \beta \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\
 &= \begin{bmatrix} \alpha \\ \beta \end{bmatrix}.
 \end{aligned}$$

The choice of $|0\rangle = [1, 0]^T$ and $|1\rangle = [0, 1]^T$ as the basis states is not the only possible one. What matters is that these vectors are orthonormal. They are orthogonal, with a mutual scalar product of $\langle 0|1\rangle = 0.0$, and normalized, with scalar products of $\langle 0|0\rangle = 1.0$ and $\langle 1|1\rangle = 1.0$.

The set of orthonormal bases $[1, 0]^T$ and $[0, 1]^T$ for the qubit vector space, which is also called the *computational basis*, is intuitive and simplifies the math. But other bases are possible, especially the ones resulting from rotations. Those are commonplace in quantum computing, as we will see shortly.

2.3 States

As we saw in the previous section, qubits are *states*, vectors of complex numbers representing probability amplitudes. We should use our trusty `Tensor` class to represent states in code. We inherit a `State` class from `Tensor` and add a moderately improved print function. The sources are in the open-source repository in `lib/state.py`:

```

class State(tensor.Tensor):
    """class State represents single and multi-qubit states."""

    def __repr__(self) -> str:
        s = 'State('
        s += super().__str__().replace('\n', '\n' + ' ' * len(s))
        s += ')'
        return s

```

```
def __str__(self) -> str:
    s = f'{self.nbits}-qubit state.'
    s += ' Tensor:\n'
    s += super().__str__()
    return s
```

The state of two or more qubits is defined as their tensor product. To compute it, we added the `*` operator to the underlying `Tensor` type in the previous section (implemented as the corresponding Python `__mul__` member function). Note that the tensor product of two states, which both have a norm of 1.0, also has a norm of 1.0.

For two qubits $|\phi\rangle$ and $|\chi\rangle$, the combined state can be written as in [Equation \(1.3\)](#), with $\otimes$ being the symbol for the Kronecker product:

$$|\psi\rangle = |\phi\rangle \otimes |\chi\rangle = |\phi\rangle|\chi\rangle = |\phi, \chi\rangle = |\phi\chi\rangle.$$

Given this definition, the state for n qubits is a `Tensor` of 2^n complex numbers, the probability amplitudes. We could maintain this length as an extra member variable to `State`, but it is easy to compute from the length of the state vector (which is already maintained by `numpy`). We define it as a property. Because this property is required for all classes derived from `Tensor` (e.g., `States` and `Operators`), we add the `nbits` property to the `Tensor` base class, so that derived classes can inherit it:

```
@property
def nbits(self) -> int:
    """Compute the number of qubits in the state."""

    return int(math.log2(self.shape[0]))
```

Python does have a `bit_length()` function to determine how many bits are needed to represent a number. Here, using this function would be wrong. To represent eight states, $n = 3$ for a state of 2^n complex numbers. However, you would need four classical binary bits to represent the number 8. Using a value of $n - 1$ will not work for an input value of 0. Additionally, `bit_length()` returns values for negative numbers, which makes no sense in this quantum state context. For all these reasons, we decided to use the `log2` function. As a code example, let us combine two qubits:

```
psi = state.qubit(alpha=1.0)    # corresponds to |0>
phi = state.qubit(beta=1.0)     # corresponds to |1>
combo = psi * phi
print(combo)

>>
2-qubit state. Tensor:
[0.+0.j 1.+0.j 0.+0.j 0.+0.j]
```

The resulting state is a `Tensor` with the complex values `[0.0, 1.0, 0.0, 0.0]`, which is the result you would expect from tensoring $[1, 0]^T$ and $[0, 1]^T$. The index of the value

2.3.1 Qubit Ordering

The ordering of qubits during construction, access to results, and conversion of binary strings are important and can be sources of problems if not understood properly. For this text, the key points to internalize are the following:

- As qubits are added to a circuit, they are being added from left to right (in a binary string), from the high-order qubit to the low-order qubit.
- In Dirac notation, two tensored states are written as $|x, y\rangle$, for example, $|0, 1\rangle$. Additionally, in this notation the most significant bit is the first to appear. States like this are also expressed as decimals. Here, the state is interpreted as $|1\rangle$ (binary 01) as opposed to $|2\rangle$ (binary 10) if incorrectly read from right to left:

$$\underbrace{|0\rangle}_{\text{High-order}} \otimes \dots \otimes \underbrace{|0\rangle}_{\text{Low-order}}$$

- We will see in [Section 2.8](#) that circuits are drawn as a vertical stack of qubits, with the top qubit being the most significant.
- We will learn soon about simple functions for constructing composite states from $|0\rangle$ and $|1\rangle$ states. In these functions, the first qubit to appear will be the most significant qubit, similar to the circuit notation. For example, the state $|\psi\rangle = |1\rangle \otimes |0\rangle \otimes |1\rangle \otimes |0\rangle$ is constructed with

```
psi = state.bitstring(1, 0, 1, 0)
```

- When states are formatted to print, the most significant bit will also be to the left, as in binary notation.
- We have to distinguish between how bits or qubits are interpreted and how we store them in our programs. In classical computing, bit 0 is typically the rightmost bit, which is the least significant bit. When we *store* bits as an array of bits, we will store them from low index to high array index. This means the index into the array is 0 for the first stored bit. In the quantum case, this is the most significant qubit.

This is a constant source of confusion, not just in this context but in any context that has to represent bits or qubits in an indexed, array-like data structure.

2.3.2 Binary Interpretation

We can write tensor products of $|0\rangle$ and $|1\rangle$ states as in this example:

$$|0\rangle \otimes |1\rangle \otimes |1\rangle = |011\rangle.$$

For brevity, when interpreting the bitstrings as binary numbers and numbering them in decimal, states are often written as in this example:

$$|011\rangle = |3\rangle.$$

Be aware of the potential for confusion between the state $|000\rangle$, the corresponding decimal state $|0\rangle$, and an actual one-qubit state $|0\rangle$. How does the decimal interpretation of a state relate to the state vector?

- The state $|00\rangle$ is computed as $\begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$. Also called $|0\rangle$.
- The state $|01\rangle$ is computed as $\begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$. Also called $|1\rangle$.
- The state $|10\rangle$ is computed as $\begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$. Also called $|2\rangle$.
- The state $|11\rangle$ is computed as $\begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$. Also called $|3\rangle$.

To find the probability amplitude for a given state, we can use binary addressing. The state vector for the three-qubit state $|011\rangle$ is:

$$[0, 0, 0, 1, 0, 0, 0, 0]^T.$$

Interpreting the right-most qubit in $|011\rangle$ as the least significant bit 0 with a bit value of 1, the middle one as bit 1 with a bit value of 1 and a power-of-2 value of 2, and the left-most as bit 2 with a value of 0, the state $|011\rangle$ computes the value 3 in decimal, state $|3\rangle$. We index the state vector as an array, as described above, from left to right, from 0 to n. Indeed, entry 3 of the state vector is set to 1. The amplitude for each state in the state vector can be found with this simple binary addressing scheme.

Note that the tensor product representation of this 3-qubit state contains the amplitudes for all eight possible states. Seven states have an amplitude of 0.0. This already hints at a potentially more efficient sparse representation.

2.3.3 Member Functions

We now introduce a few important member functions of the `State` type. This code uses functions from the `helper` module, which is described in [Section 2.4](#).

Now that we understand the order of qubits and state vector indexing, we can add functions to `State` to return the amplitude and probability of a given state. The probability is a real number, but we still have to cast it to an actual real number via `np.real` to avoid warning messages on type conflicts.

```

def ampl(self, *bits) -> np.complexfloating:
    """Return amplitude for state indexed by 'bits'."""

    idx = helper.bits2val(bits) # in helper.py
    return self[idx]

def prob(self, *bits) -> float:
    """Return probability for state indexed by 'bits'."""

    amplitude = self.ampl(*bits)
    return np.real(amplitude.conj() * amplitude)

```

We use Python parameters that are decorated with a *. This means a variable number of arguments is allowed and the parameters are *packed* as a tuple. To unpack the tuple, you have to prefix the access with a * again, as shown in the function definitions above.

As an example, for a four-qubit state, you can get the amplitude and probability for the state $|1011\rangle$ in the following way:

```

psi.ampl(1, 0, 1, 1)
psi.prob(1, 0, 1, 1)

```

The following snippet iterates over all possible states and prints the probabilities for each state:

```

for bits in helper.bitprod(4):
    print(psi.prob(*bits))

```

During algorithm development, we often want to find the one state with the highest probability. For this, we add the following convenience function, which iterates over all possible states and returns the state/probability pair with the highest probability:

```

def maxprob(self) -> (List[float], float):
    """Find state with highest probability."""

    maxbits, maxprob = [], 0.0
    for bits in helper.bitprod(self.nbits):
        cur_prob = self.prob(*bits)
        if cur_prob > maxprob:
            maxbits, maxprob = bits, cur_prob
    return maxbits, maxprob

```

As we will see later, it can become necessary to renormalize a state vector. This is done with the `normalize` member function. Note that this function assumes that the dot product is not 0.0; otherwise, this code will result in a division by zero exception:

```
def normalize(self) -> None:
    """Renormalize the state. Sum of norms==1.0."""

    dprod = np.conj(self) @ self
    if (dprod.is_close(0.0)):
        raise AssertionError('Normalizing to zero-probability state.')
    self /= np.sqrt(np.real(dprod))
```

The *phase* of a qubit is the angle obtained when converting the qubit's complex amplitude to polar coordinates. We only use this during print-outs, so we convert the phase to degrees here.

```
def phase(self, *bits) -> float:
    """Compute phase of a state from the complex amplitude."""

    amplitude = self.ampl(*bits)
    return math.degrees(cmath.phase(amplitude))
```

Finally, to help in debugging, it is always helpful to have a dumper function that lists all of a state's relevant information. By default, this function only prints the states with nonzero probability. (Set parameter `prob_only` to `False` to see all states). An optional description string can be passed in as well.

```
def state_to_string(bits) -> str:
    """Convert state to string like |010>."""

    s = ''.join(str(i) for i in bits)
    return '|{:s}> ({:d}>).format(s, int(s, 2))

def dump_state(psi, desc: Optional[str]=None,
               prob_only: bool=True) -> None:
    """Dump probabilities for a state, as well as local qubit state."""

    if desc:
        print('/', end='')
        for i in range(psi.nbits-1, -1, -1):
            print(i % 10, end='')
        print(f'> \'{desc}\''

state_list: List[str] = []
for bits in helper.bitprod(psi.nbits):
    if prob_only and (psi.prob(*bits) < 10e-6):
        continue
    state_list.append(
        '{:s}:  ampl: {:.2f} prob: {:.2f} Phase: {:.1f}'
        .format(state_to_string(bits),
                psi.ampl(*bits),
                psi.prob(*bits),
                psi.phase(*bits)))
```

```

state_list.sort()
print(*state_list, sep='\n')

def dump(self, desc: Optional[str] = None,
        prob_only: bool = True) -> None:
    dump_state(self, desc, prob_only)

```

As an example, the output from the dumper may look like the following, showing all states with nonzero probability:

```

|001> (|1>):  ampl: +0.50+0.00j prob: 0.25 Phase:  0.0
|011> (|3>):  ampl: +0.35+0.35j prob: 0.25 Phase: 45.0
|101> (|5>):  ampl: +0.00+0.50j prob: 0.25 Phase: 90.0
|111> (|7>):  ampl: -0.35+0.35j prob: 0.25 Phase: 135.0

```

2.3.4 State Constructors

Using the methods described so far, let us define standard constructors to create composite states. The first two functions are for states consisting of all $|0\rangle$ and $|1\rangle$.

```

# The functions zeros() and ones() produce the all-zero or all-one
# computational basis vector for `d` qubits, i.e.,
#     |000...0> or
#     |111...1>
#
# The result of this tensor product is
# always [1, 0, 0, ..., 0]^T or [0, 0, 0, ..., 1]^T
#
def zeros_or_ones(d: int = 1, idx: int = 0) -> State:
    """Produce the all-0/1 basis vector for `d` qubits."""

    if d < 1:
        raise ValueError('Rank must be at least 1.')
    shape = 2**d
    t = np.zeros(shape, dtype=tensor.tensor_type())
    t[idx] = 1
    return State(t)

def zeros(d: int = 1) -> State:
    """Produce state with 'd' |0>, eg., |0000>."""
    return zeros_or_ones(d, 0)

def ones(d: int = 1) -> State:
    """Produce state with 'd' |1>, eg., |1111>."""
    return zeros_or_ones(d, 2**d - 1)

```

The function `bitstring` allows the construction of states from a defined series of $|0\rangle$ and $|1\rangle$ states. As noted above, the most significant bit comes first: